Laporan Praktikum
Kecerdasan Buatan

Text Classification



Citra M.Sidabutar

42324028

Dlll Teknologi Informasi

INSTITUT TEKNOLOGI DEL
FAKULTAS VOKASI
2026/2027

Praktikum

Code

```
import pandas as pd

df = pd.read_csv('Text_classification.csv')
print(df.head())
```

Output

```
  Category                                            Message
0      ham  Go until jurong point, crazy.. Available only ...
1      ham                      Ok lar... Joking wif u oni...
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...
3      ham  U dun say so early hor... U c already then say...
4      ham  Nah I don't think he goes to usf, he lives aro...
```

Penjelasan

Kode ini digunakan untuk membaca dan melihat isi awal dataset yang akan digunakan dalam proses klasifikasi teks. Pertama, import pandas as pd memanggil pustaka Pandas untuk membantu mengelola data dalam bentuk tabel. Selanjutnya, pd.read_csv('Text_classification.csv') membaca file CSV bernama Text_classification.csv dan menyimpannya ke dalam variabel df. Setelah itu, print(df.head()) menampilkan lima baris pertama dari dataset agar kita dapat melihat contoh isi data, nama kolom, serta format informasi yang tersedia sebelum melanjutkan ke tahap pengolahan dan pelatihan model.

Code

```
# Download resourcce NLTK yang diperlukan
import nltk
nltk.download('punkt_tab')
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

#import library yang preprocessing
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Inisialisasi lemmatizer
lemmatizer = WordNetLemmatizer()

# Fungsi preprocessing teks
Tabnine | Edit | Test | Explain | Document
def clean_text(text):
    text = text.lower()     # ubah menjadi lowercase
    text = re.sub(r'\d+', '', text)  # Hapus angka
    text = text.translate(str.maketrans('', '', string.punctuation))  # Hapus tanda baca
    words = word_tokenize(text)    # Tokenisasi
    words = [word for word in words if word not in stopwords.words('english')]   # Hapus stopwords
    words = [lemmatizer.lemmatize(word) for word in words]   # Lemmatization
    return ' '.join(words)

# Terapkan preprocessing ke teks
df['clean_text'] = df['Message'].apply(clean_text)

# Cek hasil preprocessing
print("\nHasil Preprocessing:\n", df[['Message', 'clean_text']].head())
```

Output

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\acer\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\acer\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\acer\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\acer\AppData\Roaming\nltk_data...

Hasil Preprocessing:
                                              Message  \
0  Go until jurong point, crazy.. Available only ...
1                      Ok lar... Joking wif u oni...
2  Free entry in 2 a wkly comp to win FA Cup fina...
3  U dun say so early hor... U c already then say...
4  Nah I don't think he goes to usf, he lives aro...

                                         clean_text
0  go jurong point crazy available bugis n great ...
1                            ok lar joking wif u oni
2  free entry wkly comp win fa cup final tkts st ...
3                u dun say early hor u c already say
4           nah dont think go usf life around though
```

Penjelasan

Kode ini berfungsi untuk membersihkan teks sebelum data diolah oleh model klasifikasi. Pertama-tama, pustaka NLTK mengunduh beberapa bahan penting, seperti tokenizer, stopwords, dan WordNet untuk mendukung pengolahan kata. Selanjutnya, pustaka re dan string dimanfaatkan untuk menghilangkan angka serta tanda baca, sementara word_tokenize, stopwords, dan WordNetLemmatizer berfungsi untuk memecah kalimat menjadi kata-kata, menghapus istilah umum dalam bahasa Inggris yang tidak memberikan banyak informasi, serta mengubah kata-kata ke bentuk dasarnya. Fungsi clean_text() kemudian melaksanakan langkah-langkah tersebut secara berurutan, dimulai dengan mengonversi semua huruf menjadi huruf kecil, menghapus angka dan tanda baca, memecah teks ke dalam kata-kata, mengeliminasi stopwords, serta melakukan lemmatization. Setelah itu, df['Message']. apply(clean_text) menerapkan fungsi pembersihan ini pada setiap pesan yang ada di kolom Message, kemudian menyimpan hasilnya di kolom baru yang diberi nama clean_text. Terakhir, print() menampilkan lima contoh pesan asli beserta hasil pembersihan sehingga perubahan pada teks dapat diperiksa sebelum dipakai dalam proses klasifikasi.

Code

```
# Ubah Label dari "ham / spam" menjadi 0/1
df['Category'] = df['Category'].map({'ham': 0, 'spam': 1})

# Cek jumlah masing-masing kelas
print(df['Category'].value_counts())
```

Output

```
Category
0    4825
1     747
Name: count, dtype: int64
```

Penjelasan

Kode ini digunakan untuk mengubah label kategori pesan dari bentuk teks menjadi angka agar lebih mudah diproses oleh model machine learning. Pada bagian df['Category'].map({'ham': 0, 'spam': 1}), label ham diubah menjadi 0 yang menunjukkan pesan normal, sedangkan label spam diubah menjadi 1 yang menunjukkan pesan spam. Hasil perubahan tersebut disimpan kembali ke dalam kolom Category. Selanjutnya, df['Category'].value_counts() digunakan untuk menghitung jumlah data pada setiap kategori, sehingga kita dapat mengetahui banyaknya pesan normal dan pesan spam dalam dataset yang digunakan.

Code

```
from sklearn.model_selection import train_test_split

# Pisahkan fitur dan Label
X = df['clean_text']
y = df['Category']

# Split data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Penjelasan

Kode ini dimaksudkan untuk mengkonversi label kategori pesan dari format teks ke angka sehingga lebih mudah untuk diproses oleh model machine learning. Pada bagian df['Category']. map({'ham': 0, 'spam': 1}), label ham diubah menjadi 0 yang menunjukkan pesan yang bukan spam, sementara label spam diubah menjadi 1 yang menunjukkan pesan yang terindikasi spam. Hasil dari konversi ini disimpan kembali ke dalam kolom Category. Selanjutnya, df['Category']. value_counts() digunakan untuk menghitung jumlah data dalam setiap kategori, sehingga kita dapat mengetahui total pesan yang termasuk dalam kategori normal dan kategori spam dalam dataset yang sedang digunakan.

Code

```python
from sklearn.feature_extraction.text import TfidfVectorizer

# Konversi teks menjadi fitur numerik menggunakan TF-IDF
vectorizer = TfidfVectorizer(ngram_range=(1,2), max_features=5000) # Menggunakan bigram
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

print("\nTF-IDF Shape (Train):", X_train_tfidf.shape)
```

Output

```
TF-IDF Shape (Train): (4457, 5000)
```

Penjelasan

Kode ini berfungsi untuk mengonversi teks menjadi atribut numerik dengan teknik Term Frequency-Inverse Document Frequency (TF-IDF), memungkinkan model machine learning untuk memahami kata-kata dalam pesan. Pertama, TfidfVectorizer diambil dari Scikit-learn, lalu diatur dengan ngram_range=(1,2) agar sistem dapat memasukkan kata tunggal (unigram) dan pasangan dua kata yang berdampingan (bigram), serta max_features=5000 untuk mengatur batas maksimal jumlah fitur yang boleh digunakan. Berikutnya, vectorizer. fit_transform(X_train) menganalisis kosakata dari dataset pelatihan sekaligus mendigitalkan teks menjadi representasi angka, sedangkan vectorizer. transform(X_test) mengonversi data pengujian dengan kosakata yang sama tanpa menganalisis fitur dari data tersebut secara berulang. Akhirnya, X_train_tfidf. shape menunjukkan dimensi data TF-IDF dari pelatihan, merinci jumlah pesan dan jumlah fitur yang dihasilkan, sehingga kita dapat mengerti seberapa banyak data dan fitur yang akan dimanfaatkan oleh model.

Code

```python
from sklearn.naive_bayes import MultinomialNB

# Inisialisasi model Naïve Bayes
model = MultinomialNB(alpha=0.1)

# Train model
model.fit(X_train_tfidf, y_train)

# Prediksi hasil
y_pred = model.predict(X_test_tfidf)
```

Penjelasan

Dalam kode ini, algoritma Multinomial Naïve Bayes diterapkan untuk menganalisis pola kata yang terdapat dalam pesan spam serta pesan yang biasa. Parameter alpha=0.1 berfungsi untuk mengatasi isu probabilitas nol saat ada kata yang tidak muncul dalam data pelatihan. Proses pembelajaran dilakukan dengan menggunakan model. fit(X_train_tfidf, y_train), memanfaatkan data teks yang telah diubah ke dalam format nilai TF-IDF serta label kategorinya. Setelah model dilatih, baris model. predict(X_test_tfidf) digunakan untuk mengklasifikasikan kategori pesan dalam data

pengujian berdasarkan pola yang telah dipelajari sebelumnya. Hasil klasifikasi tersebut disimpan pada variabel y_pred dan selanjutnya dapat dimanfaatkan untuk mengevaluasi seberapa efektif model dalam mengenali pesan spam dan pesan biasa.

Code

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Akurasi
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Classification Report
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output

```
Accuracy: 0.979372197309417

Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       966
           1       0.98      0.86      0.92       149

    accuracy                           0.98      1115
   macro avg       0.98      0.93      0.95      1115
weighted avg       0.98      0.98      0.98      1115
```

Penjelasan

Kode ini berfungsi untuk menilai performa model dalam mengidentifikasi pesan spam dan bukan spam. Pertama, program mengambil accuracy_score, classification_report, dan confusion_matrix dari Scikit-learn untuk mendukung proses evaluasi model. Kemudian, accuracy_score(y_test, y_pred) dipakai untuk menghitung akurasi dengan membandingkan hasil prediksi model dengan label asli dalam data uji. Nilai akurasi yang diperoleh disimpan dalam variabel accuracy dan ditampilkan menggunakan print(). Setelah itu, classification_report(y_test, y_pred) digunakan untuk memberikan laporan evaluasi yang mencakup precision, recall, f1-score, dan support untuk setiap kategori. Precision menggambarkan keakuratan model saat memprediksi kategori tertentu, recall menandakan kemampuan model dalam mendeteksi data yang benar-benar sesuai kategori tersebut, f1-score merupakan titik keseimbangan antara precision dan recall, sedangkan support menunjukkan jumlah data yang benar dalam setiap kategori. Dengan demikian, kode ini memberikan gambaran seberapa efektif model dalam mengklasifikasikan pesan spam dan bukan spam berdasarkan data yang diuji.

Code

```python
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix

# Hitung confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

# Definisi elemen dalam confusion matrix
TN, FP, FN, TP = conf_matrix.ravel()

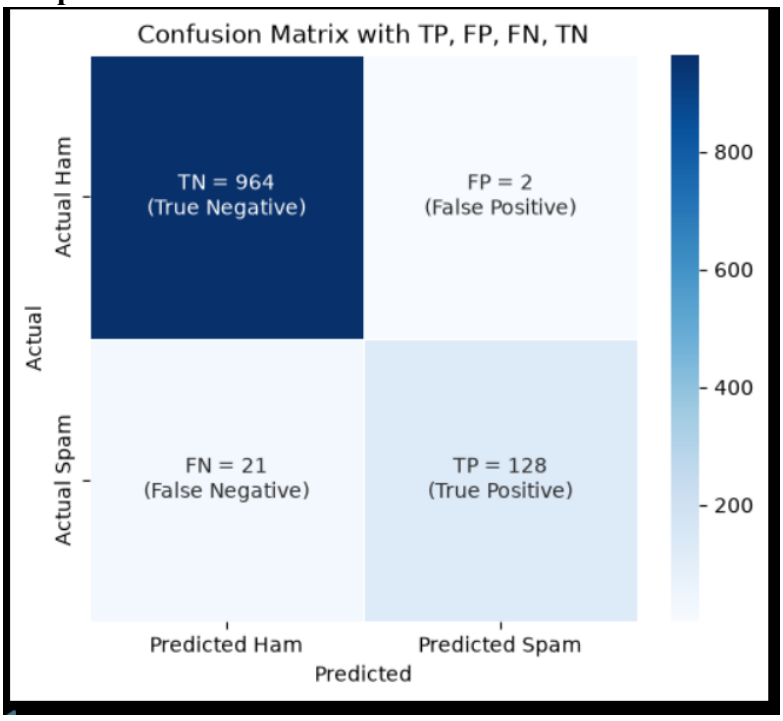
# Buat dataframe confusion matrix
df_conf_matrix = pd.DataFrame(conf_matrix, index=['Actual Ham', 'Actual Spam'], columns=['Predicted Ham', 'Predicted Spam'])

# Label untuk TP, FP, FN, TN
labels = np.array([["TN = {}\n(True Negative)".format(TN), "FP = {}\n(False Positive)".format(FP)],
                   ["FN = {}\n(False Negative)".format(FN), "TP = {}\n(True Positive)".format(TP)]])

# Plot heatmap dengan keterangan
plt.figure(figsize=(6,5))
ax = sns.heatmap(df_conf_matrix, annot=labels, fmt='', cmap='Blues', linewidths=0.5, cbar=True)

# Tambahkan Label
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix with TP, FP, FN, TN')
plt.show()
```

Output



Penjelasan

Kode ini berfungsi untuk memvisualisasikan matriks kebingungan dalam rangka menilai performa model dalam mengkategorikan pesan spam dan normal. Pertama-tama, beberapa pustaka diimpor, termasuk seaborn dan matplotlib. pyplot untuk menyajikan visual, numpy dan pandas untuk proses data, serta confusion_matrix dari Scikit-learn

untuk menghitung dan menilai performa model. Selanjutnya, confusion_matrix(y_test, y_pred) dipakai untuk mengkontraskan kategori yang sebenarnya dengan hasil ramalan dari model. Hasilnya dibagi menjadi empat komponen menggunakan conf_matrix. ravel(), yaitu TN (True Negative) yang menunjukkan bahwa pesan biasa diidentifikasi sebagai ham, FP (False Positive) yang merujuk pada pesan biasa yang keliru diidentifikasi sebagai spam, FN (False Negative) yang menandakan pesan spam yang salah diidentifikasi sebagai ham, dan TP (True Positive) yang menunjukkan pesan spam yang berhasil dikenali sebagai spam. Setelah itu, pd. DataFrame() diterapkan untuk menamai baris dan kolom agar hasilnya lebih jelas dibaca, sedangkan np.array() digunakan untuk membuat label yang mencakup angka TN, FP, FN, dan TP serta penjelasannya. Kemudian, plt.figure() digunakan untuk menetapkan ukuran gambar, dan sns.heatmap() menyajikan hasil matriks kebingungan dalam format peta panas dengan angka dan penjelasan di setiap bagiannya. Akhirnya, plt.xlabel(), plt.ylabel(), dan plt.title() dimanfaatkan untuk menambahkan label pada sumbu serta judul, di mana plt.show() akan menampilkan visualisasinya sehingga kita bisa menilai seberapa efektif model dalam membedakan antara pesan spam dan normal.

Code

```
# contoh teks baru
new_texts = ["Congratulations! You won a free trip to Bali.",
             "Hey, are you coming to the party tonight?"]

# Preprocess teks baru
new_texts_clean = [clean_text(text) for text in new_texts]

# Transformasi dengan TF-IDF
new_texts_tfidf = vectorizer.transform(new_texts_clean)

# Prediksi
predictions = model.predict(new_texts_tfidf)

# Output hasil prediksi
for text, label in zip(new_texts, predictions):
    print(f"\nMessage: {text} \nPredicted: {'Spam' if label == 1 else 'Ham'}")
```

Output

```
Message: Congratulations! You won a free trip to Bali.
Predicted: Spam

Message: Hey, are you coming to the party tonight?
Predicted: Ham
```

Penjelasan

Kode diatas ini berfungsi untuk menguji model yang telah dilatih dalam mengkategorikan pesan baru ke dalam klasifikasi spam atau ham. Pertama-tama, variabel new_texts berisi dua contoh pesan yang akan diuji, yaitu sebuah tawaran perjalanan gratis menuju Bali dan pertanyaan mengenai kehadiran seseorang di acara malam. Selanjutnya, fungsi clean_text() dipakai untuk memproses teks agar sesuai dengan format data saat proses pelatihan. Setelah itu, vectorizer. transform() mengkonversi teks yang sudah dibersihkan

menjadi representasi numerik menggunakan teknik TF-IDF agar model dapat memprosesnya. Selanjutnya, model. predict() dipakai untuk menentukan kategori setiap pesan berdasarkan pola yang telah dipelajari sebelumnya. Akhirnya, perulangan for digunakan untuk menampilkan pesan berserta hasil prediksinya. Jika nilai prediksi menunjukkan 1, pesan tersebut diklasifikasikan sebagai spam, sedangkan jika nilainya 0, pesan tersebut dianggap sebagai ham atau pesan biasa. Hasil prediksi ini bergantung pada pola yang didapat oleh model selama tahap pelatihan.